



TEORÍA DE ALGORITMOS
CURSO ECHEVARRIA

Trabajo Práctico 1

May 31, 2025

- Juan Claros - 111254
- Yoel Gaston Arlia -
111520
- Guido Cuppari -
111172
- Josue Daniel Mamani
- 111514

Índice

1	Problema 1	3
1.1	Supuestos	3
1.2	Diseño	3
1.3	Pseudocodigo	3
1.4	Análisis de Complejidad	4
1.5	Seguimiento	4
1.6	Casos de prueba y mediciones	5
1.7	Resultados	5
1.8	Generación de sets	5

1 Problema 1

1.1 Supuestos

- Se supone que el algoritmo recibe una cadena con los palindromos, y el algoritmo buscara la minima cantidad de palindromos posibles que forma la cadena dada por parametro, mediante programacion dinamica.

1.2 Diseño

- La ecuacion de recurrencia a la que se llego es: $opt[i] = 1$ si $palabra[i :]$ es palindromo y $opt[i] = \min(opt[j + 1] + 1)$ para todo j tal que $palabra[i : j + 1]$ es palindromo.
- Subestructura óptima: La solución óptima para el prefijo $palabra[0 : i]$ depende de las soluciones óptimas de los sufijos $palabra[j + 1 :]$, que son subproblemas del problema original.

Subproblemas superpuestos: Evaluar si una subcadena es palíndromo se repite muchas veces. Por eso se usa una tabla booleana `es_palindromo[i][j]` para memorizar los resultados.

- Se usa memoization en 2 estructuras de datos, para guardar en un arreglo de enteros (todos infinitos positivos de base) $opt[i]$ el minimo numero de palindromos para $palabra[i :]$. Y para guardar en una matriz booleana (con todas las posiciones false de base) de $n \times n$ `es_palindromo[i][j]` si $palabra[i : j + 1]$ es un palindromo.

1.3 Pseudocodigo

```
1 def min_palindromos(palabra):
2     n = len(palabra)
3
4     opt = [float('inf')] * (n + 1)
5     opt[n] = 0
6
7     es_palindromo = [[False] * n for _ in range(n)]
8
9     for i in range(n - 1, -1, -1):
10         for j in range(i, n):
11             if palabra[i] == palabra[j] and (j - i <= 1 or
12             es_palindromo[i + 1][j - 1]):
13                 es_palindromo[i][j] = True
14
15         for i in range(n - 1, -1, -1):
16             for j in range(i, n):
17                 if es_palindromo[i][j]:
18                     opt[i] = min(opt[i], 1 + opt[j + 1])
19
20     return opt[0]
```

```

21 cadena = "ALLAERRESAGASRADAR"
22 print(min_palindromos(cadena))
23

```

1.4 Analisis de Complejidad

- Los puntos clave para analizar la complejidad de este algoritmo son los 2 for anidados que hay tanto para el procesamiento de los palindromos que se ejecuta para cada par i, j en:

```

1     for i in range(n - 1, -1, -1):
2         for j in range(i, n):
3             if palabra[i] == palabra[j] and (j - i <= 1 or
4             es_palindromo[i + 1][j - 1]):
5                 es_palindromo[i][j] = True

```

Y para la construccion de la tabla booleana $opt[i][j]$ que tambien revisa cada subcadena $palabra[i : j + 1]$, y si la misma es palindromo realiza una operacion constante en:

```

1     for i in range(n - 1, -1, -1):
2         for j in range(i, n):
3             if es_palindromo[i][j]:
4                 opt[i] = min(opt[i], 1 + opt[j + 1])
5

```

Por lo tanto, en ambos for anidados, el peor caso implica una doble iteracion sobre n , por ende la complejidad temporal total del algoritmo es de $O(N^2)$.

1.5 Seguimiento

- Vamos a hacer el seguimiento para el caso de la cadena "ARACALACANA": lo primero que hace el algoritmo es obtener el largo de la cadena recibida mediante $len()$, y procede a generar las 2 estructuras utilizadas, la lista de enteros y la matriz de booleanos, para luego llegar a la primer parte importante, el pre-procesamiento de los palindromos, donde se marca como True toda subcadena de $palabra[i : j + 1]$ que sea un palindromo.
- Para ver si un substring es palindromo o no, se utilizan 3 factores, si $palabra[i] = palabra[j]$ (la palabra empieza y termina con la misma letra) y si $j - i \leq 1$ (es de 1 o 2 caracteres) o $es_palindromo[i + 1][j - 1] = True$. Entonces empieza desde $i = 10$ y siempre lo primero que llena en cada i es la diagonal, que es trivialmente un palindromo, al ser un solo caracter, pero empieza llenando la matriz desde el final.
- La primer posicion analizada es $[10][10]$, siendo la cadena "A", que es un caso trivial, cumpliendo la condicion y llenando la tabla en esa posicion con True, luego sigue $[9][9]$, que ocurre lo mismo que en el caso anterior, siendo simplemente el caracter "N", $[9][10]$ no cumple la condicion, ya que no empieza

y termina con la misma letra, quedandose en False..., saltamos al ejemplo del siguiente palindromo no trivial, que es [8][10] la cadena "ANA" cumple que empieza y termina con la misma letra, no tiene menos de 2 caracteres, pero `es_palindromo[i + 1][j - 1]` que en este caso es [9][9] es True, entonces rellena True dandola como palindromo.

- Para el final de las iteraciones el algoritmo rellena la tabla ubicando True en todos los palindromos que encuentra.
- Una vez rellena la tabla, pasa a la construccion de `opt[i]` buscando la minima cantidad de palindromos de la palabra, donde `opt[n] = 0`, siendo el caso base la string vacia, luego empieza la iteracion desde $i = 10$ hasta 0.
- Entonces empieza con [10][10], le pregunta a la tabla si es palindromo, en este caso lo es, por ende `opt[i]` es igual al minimo entre si mismo, que de base es infinito, y $1 + \text{opt}[j+1]$, que en este caso es $1 + \text{opt}[11]$ (que es 0), entonces `opt[10] = 1`. Sigue con $i = 9, j = 9$, se le pregunta si la [9][9] es True, lo cual es correcto, entonces se usa la ecuacion siendo `opt[9] = min(opt[9], 1 + opt[10])`, que termina siendo `opt[9] = min(infinito, 2)`, `opt[9] = 2`, y asi sigue con toda la tabla para finalmente encontrar el optimo que es 3.

1.6 Casos de prueba y mediciones

1.7 Resultados

- Los resultados, segun la cantidad de particulas con las cantidades de 100, 200, 400, 600, 800 y 1000, son los esperados y coinciden con la complejidad.

1.8 Generacion de sets

- Los sets son realizados a partir de una funcion randomizada, que generan archivos de texto con listas que representan las cargas a probar.